# CS-112
# Object Oriented Programming

DEPARTMENT OF COMPUTER SCIENCE

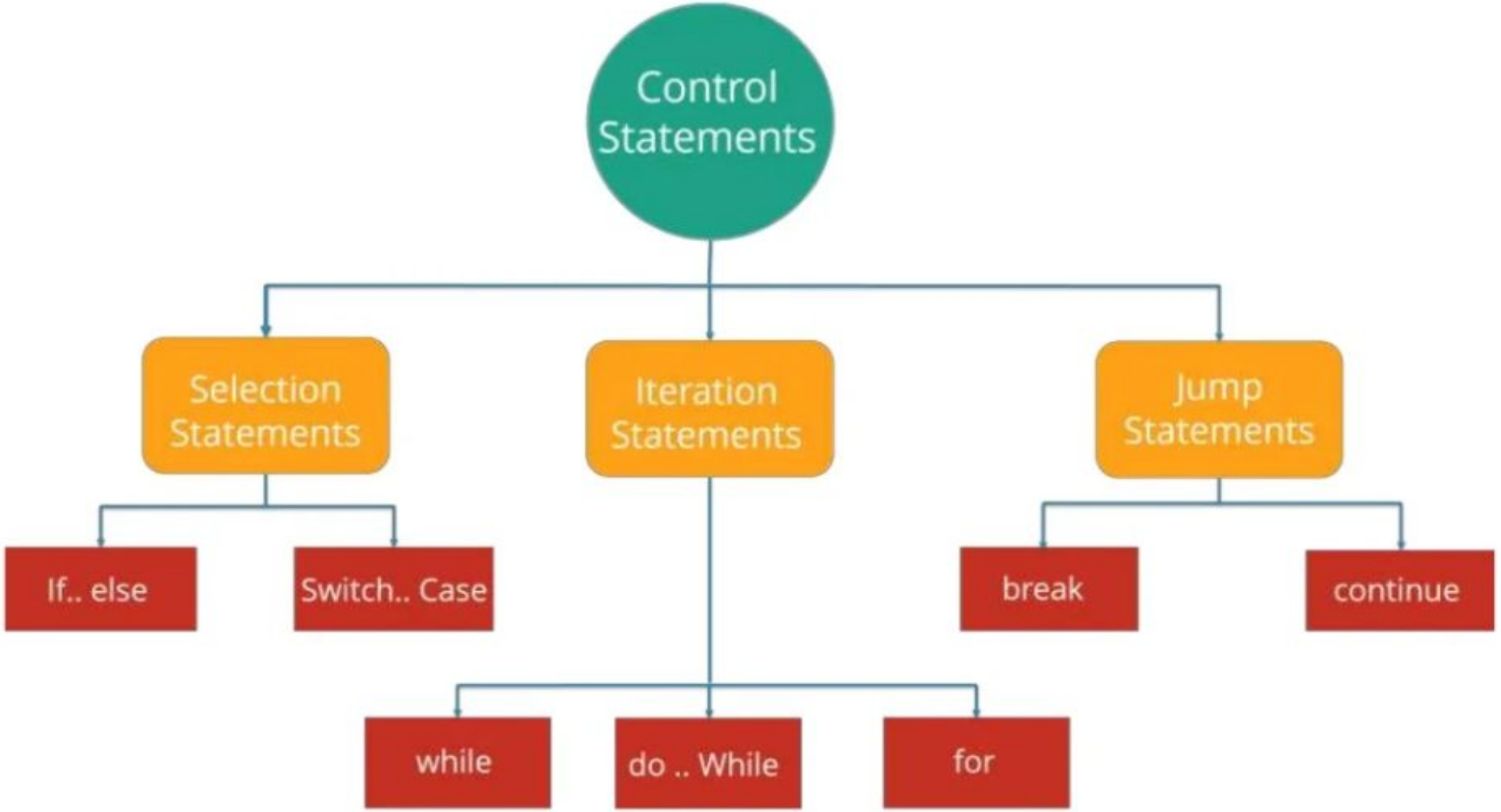
COURSE INSTRUCTOR : MS. HABIBA ARSHAD

# Control Structures in Java

# Control Statements

- The control statement are used to **control the flow of execution** of the program.
- This execution order depends on the supplied **data values** and the **conditional logic**.
- In java program, control structure is can divide in three parts:

1. **Selection statement**
2. **Iteration statement**
3. **Jumps in statement**


Control Statements
Selection Statements
Iteration Statements
Jump Statements
If.. else
Switch.. Case
while
do .. While
for
break
continue

# Selection Statement

- Selection statement is also called as **Decision making** statements.
- Because it provides the decision making capabilities to the statements.
- In selection statement, there are two types:

1. **if statement**
2. **Switch statement.**

# Java If Statement

- The Java *if statement* is used to **test the condition**.
- It checks Boolean condition: ***true* or *false*.**
- There are various types of if statement in java.

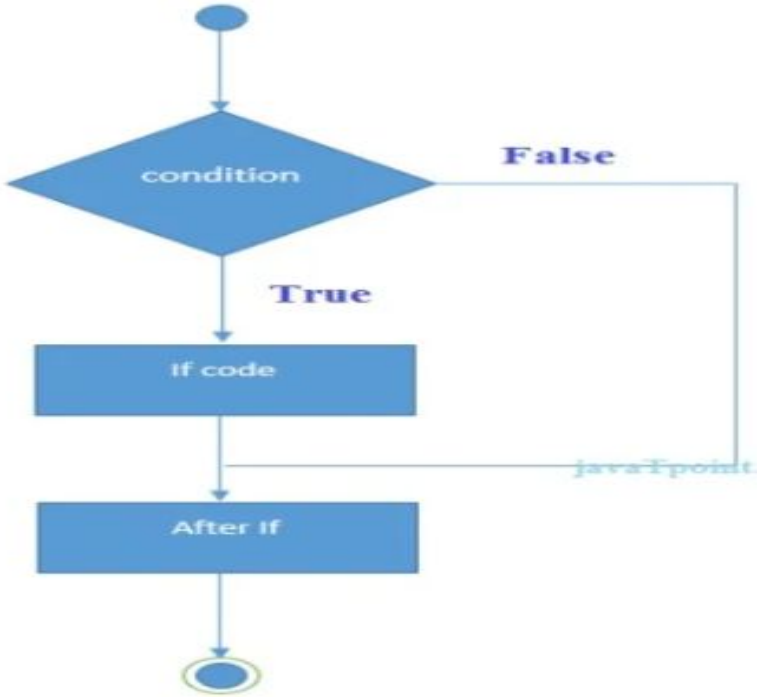
1. if statement
2. if-else statement
3. Nested if-else statement
4. if-else-if ladder

# IF Statement

- The Java if statement tests the condition.
- It executes the ***if block*** if condition is **true**.

Syntax:

```
if(condition)
{
//code to be executed
}
```



## Example:

```
public class IfExample
{
public static void main(String[] arg)
{
 int age=20;
   if(age>18)
{
      System.out.print("Age is greater than 18");
   }
}}
```

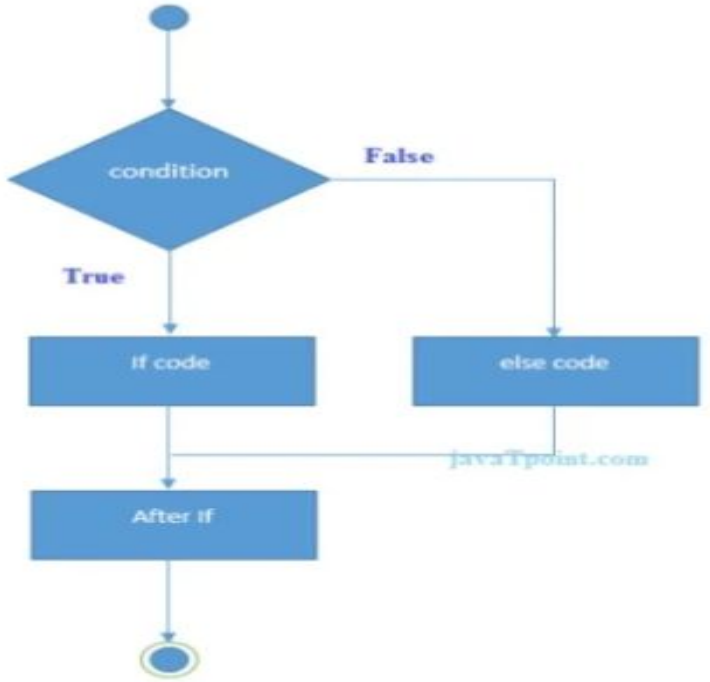
**Output:** Age is greater than 18

# IF Else Statement

- The Java if-else statement also tests the condition.
- It executes the *if block* if condition is true otherwise *else block* is executed.

**Syntax:**

```
if(condition)
{
//code if condition is true
}
Else
{
//code if condition is false
}
```



```
public class IfElseExample
{
public static void main(String[] args)
 {
   int number=13;
   if(number%2==0)
{
      System.out.println("even number");
}
Else
{
      System.out.println("odd number");
   }}}
```

**Output:** odd number

# Nested If Statement

- The Nested if-else statement executes one if or else if statement inside another if or else if statement.

**Syntax:**

```
if(Boolean_expression 1)
{
// Executes when the Boolean expression 1 is true
if(Boolean_expression 2)
{
// Executes when the Boolean expression 2 is true
}
}
```

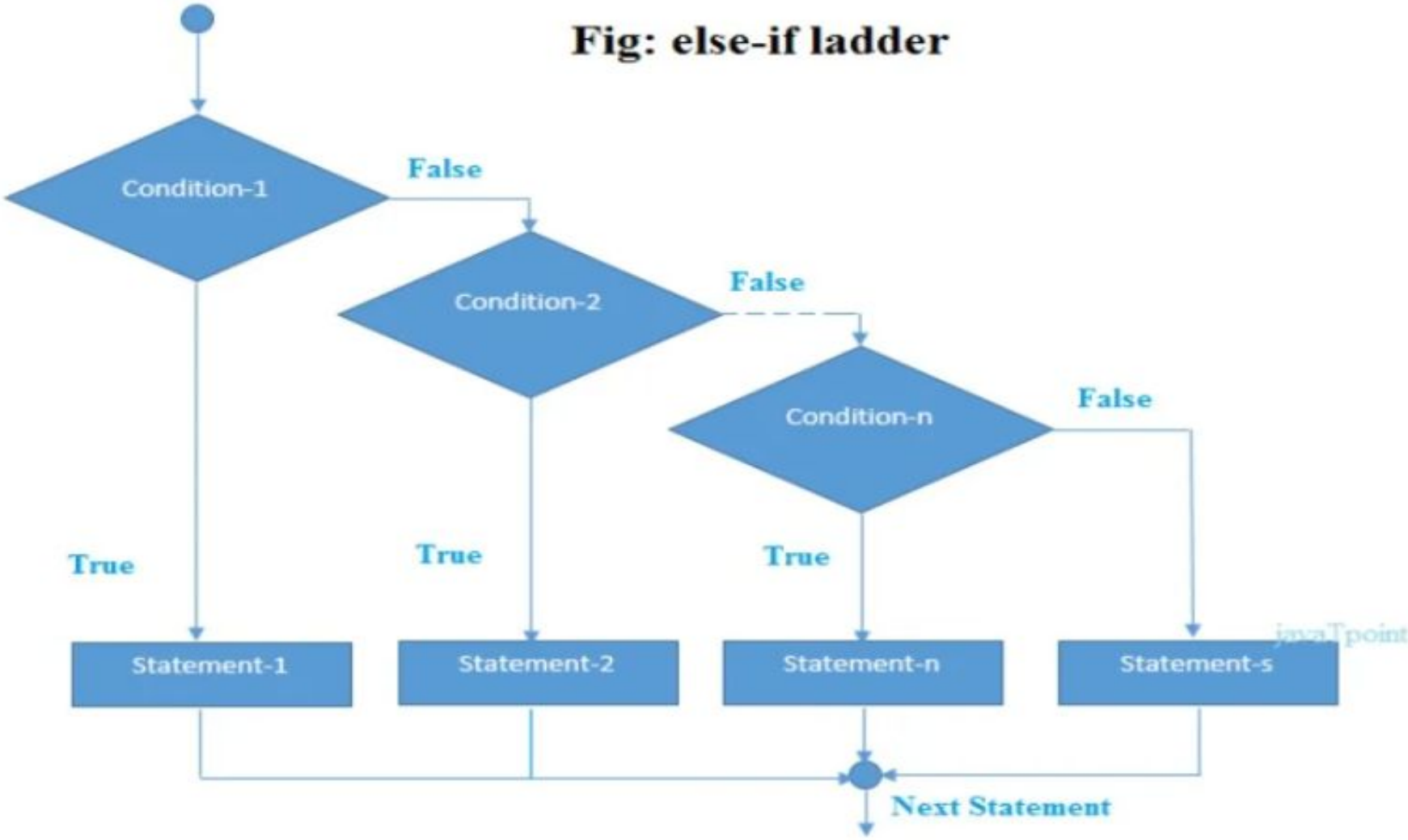
# IF – ELSE – IF Ladder

- The if-else-if ladder statement executes **one condition from multiple statements.**

**Syntax:**

```
if(condition1)
{
//code to be executed if condition1 is true
}
else if(condition2)
{
//code to be executed if condition2 is true
}
else if(condition3)
{
//code to be executed if condition3 is true
}
...
Else
{
//code to be executed if all the conditions are false
}
```

**Fig: else-if ladder**



# Example

**Write a program that inputs test score of a student and displays his grade according to the following criteria**

| Test Score | Grade |
|---|---|
| >=90 | A |
| 80-89 | B |
| 70-79 | C |
| 60-69 | D |
| Below 60 | F |

# IF – Else- IF Ladder

```
public class IfElseIfExample
{
public static void main(String[] args)
{
   int marks=65;
   if(marks<50)
{
      System.out.println("fail");
   }
   else if(marks>=50 && marks<60)
{
      System.out.println("D grade");
   }
   else if(marks>=60 && marks<70)
{
      System.out.println("C grade");
   }
else if(marks>=70 && marks<80)
{
      System.out.println("B grade");
   }
   else if(marks>=80 && marks<90)
{
      System.out.println("A grade");
   }
else if(marks>=90 && marks<100)
{
      System.out.println("A+ grade");
   }
else
{
      System.out.println("Invalid!");
   }}}
```

**Output:** C grade

# Switch Statement

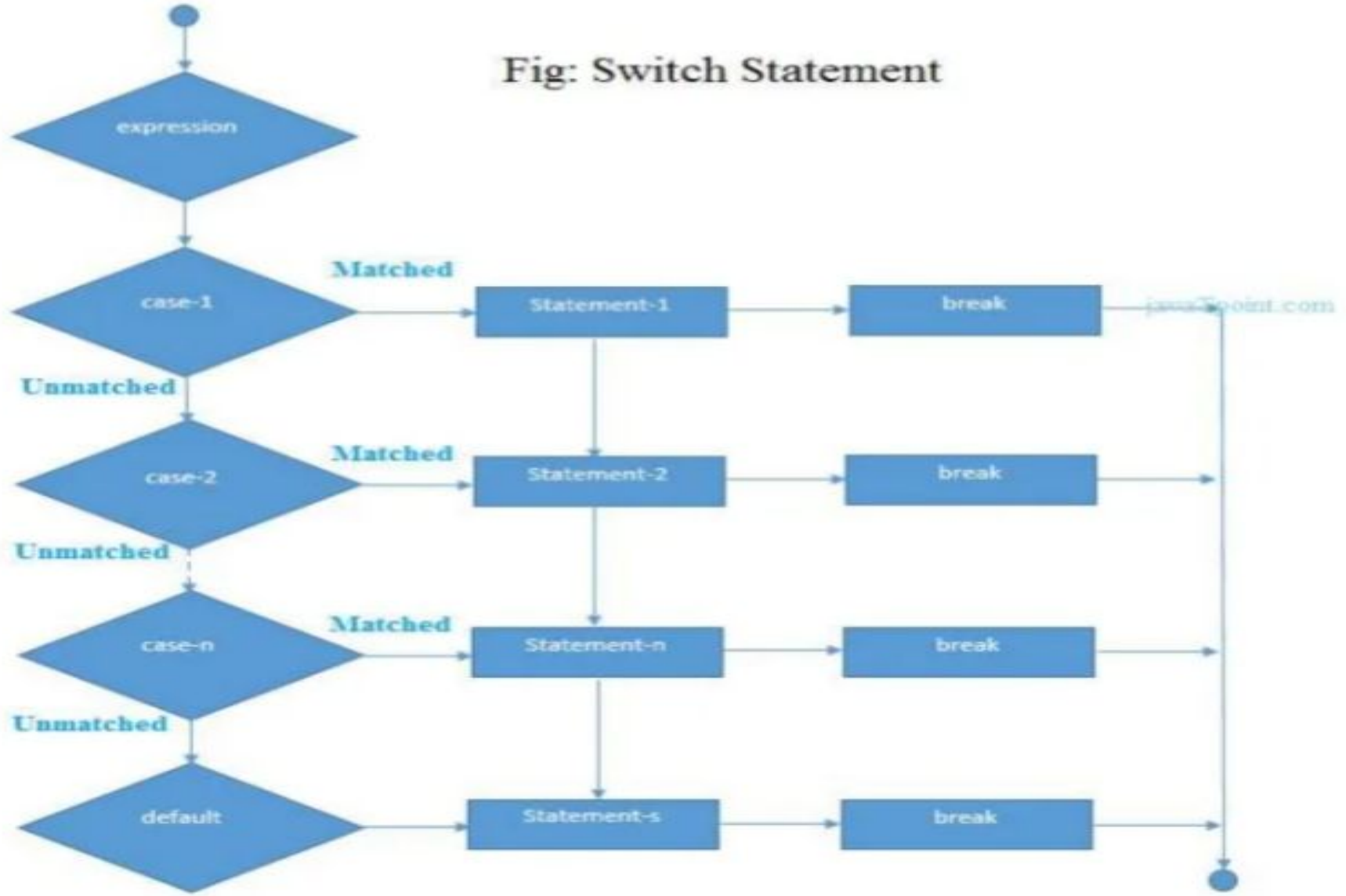
- The Java *switch statement* executes **one statement from multiple conditions.**
- It is like if-else-if ladder statement.

**Syntax:**

```
switch(expression)
{
case value1:
 //code to be executed;
 break;  //optional
case value2:
 //code to be executed;
 break;  //optional
.....

default:
 code to be executed if all cases are not matched;
}
```

Fig: Switch Statement



# Switch Statement Example

```
public class SwitchExample
{
public static void main(String[] args)
{
    int number=20;
 switch(number)
{
    case 10: System.out.println("10");break;
    case 20: System.out.println("20");break;
    case 30: System.out.println("30");break;
    default:System.out.println("Not in 10, 20 or 30");
    }}}
```

**Output**: 20

# Iteration Statement

- Looping is also called as **iterations**.
- The process of **repeatedly executing a statements** and is called as looping.
- The statements may be executed multiple times.
- If a loop executing continuous then it is called as **Infinite loop**.
- In Iteration statement, there are three types of operation:

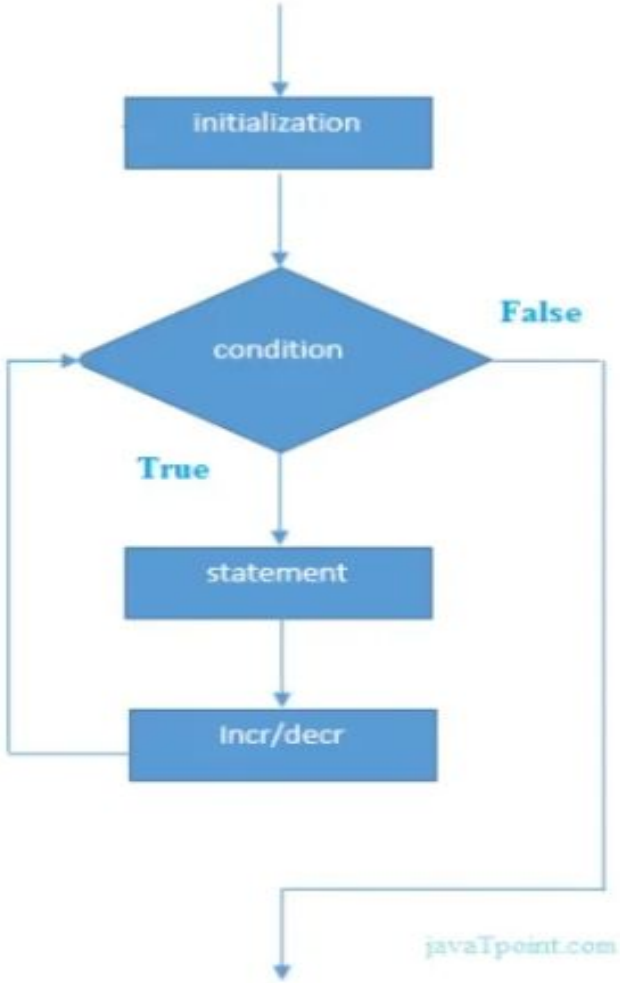
1. **for loop**
2. **while loop**
3. **do-while loop**

# FOR LOOP

- The simple for loop is same as C/C++.
- We can initialize variable, check condition and increment/decrement value.

**Syntax:**

```
for(initialization;condition;incr/decr)
{
//code to be executed
}
```



# FOR Loop Example

```
public class ForExample
{
public static void main(String[] args)
 {
   for(int i=1;i<=10;i++)
{
       System.out.print(i);
   }
}
}
```

**Output:**

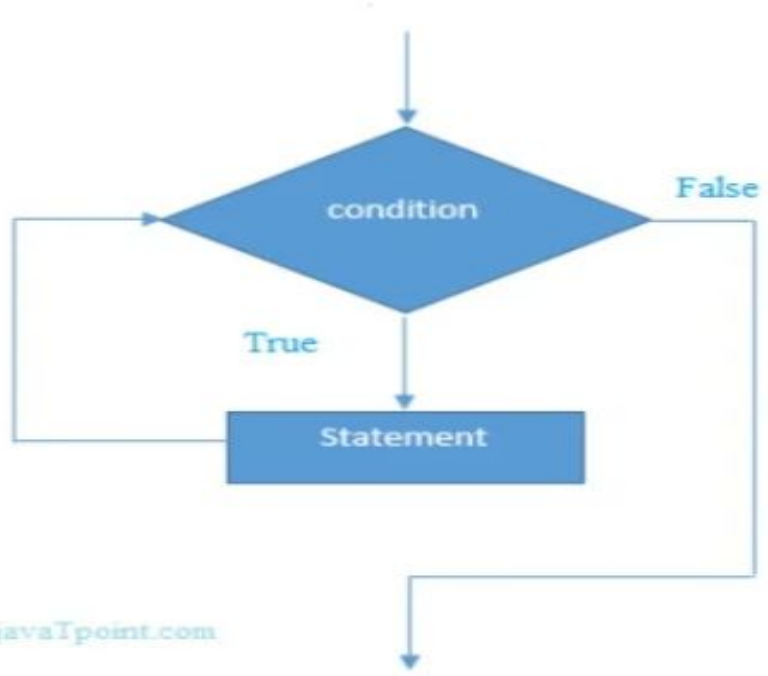
12345678910

# While Loop Example

- The Java *while loop* is used to iterate a part of the program several times.
- If the number of iteration is not fixed, it is recommended to use while loop.

**Syntax:**

```
while(condition)
{
//code to be executed
}
```



# While Loop Example

```
public class WhileExample
{
public static void main(String[] args)
{
    int i=1;
    while(i<=10)
{
    System.out.println(i);
    i++;
    }
} }
```

**Output:**

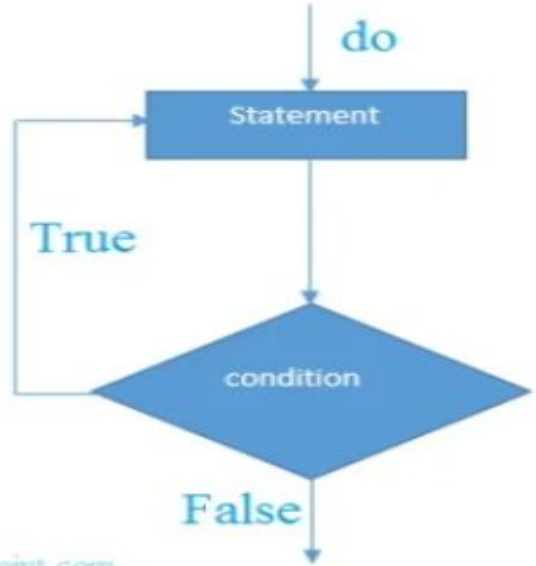
12345678910

# Do- While Loop

- The Java *do-while loop* is used to **iterate** a part of the program **several times**.
- If the number of **iteration is not fixed** and you must have to execute the loop at least once, it is recommended to use do-while loop.
- The Java *do-while loop* is executed at least once because Condition is checked **after loop body**.

**Syntax:**

```
Do
{
//code to be executed
}
while(condition);
```



# Jump Statements

Java jump statements enable transfer of control to other parts of program.

Java provides three jump statements:

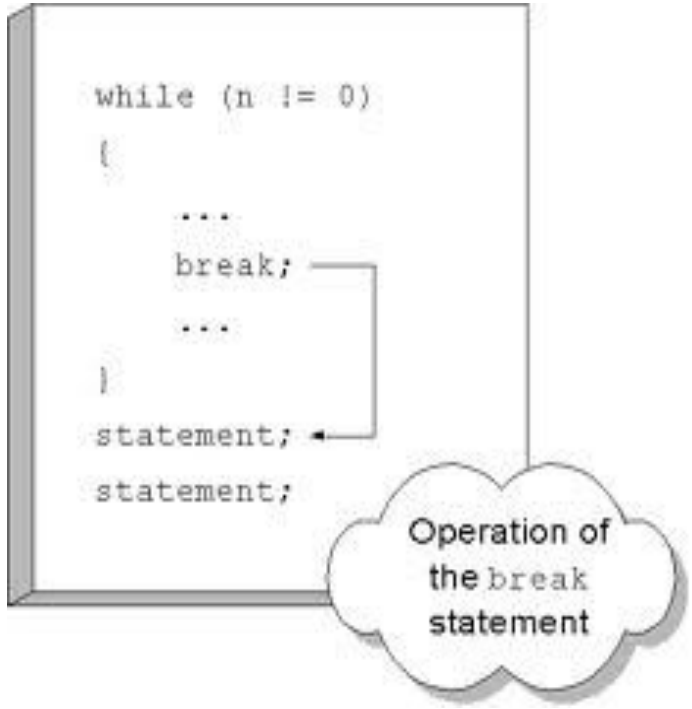
- break
- continue
- return

In addition, Java supports exception handling that can also alter the control flow of a program. Exception handling will be explained in its own section later.

# Break Statement

The break statement has three uses:

1. to terminate a case inside the switch statement
2. to exit an iterative statement
3. to transfer control to another statement



```java
//Java Program to demonstrate the use of break statement
//inside the for loop.
public class BreakExample {
public static void main(String[] args) {
    //using for loop
    for(int i=1;i<=10;i++){
        if(i==5){
            //breaking the loop
            break;
        }
        System.out.println(i);
    }
}
}
```

# Loop Exit with Break

When break is used inside a loop, the loop terminates and control is transferred to the following instruction.

# Continue Statement

The break statement terminates the block of code, in particular it terminates the execution of an iterative statement.

The continue statement is used in loop control structure when you need to jump to the next iteration of the loop immediately. It can be used with for loop or while loop.

The Java continue statement is used to continue the loop. It continues the current flow of the program and skips the remaining code at the specified condition. In case of an inner loop, it continues the inner loop only.

```java
//Java Program to demonstrate the use of continue statement
//inside the for loop.
public class ContinueExample {
public static void main(String[] args) {
    //for loop
    for(int i=1;i<=10;i++){
        if(i==5){
            //using continue statement
            continue;//it will skip the rest statement
        }
        System.out.println(i);
    }
}
}
```

# Continue Statement Example

# Return statement

The return statement is used to return from the current method: it causes program control to transfer back to the caller of the method.

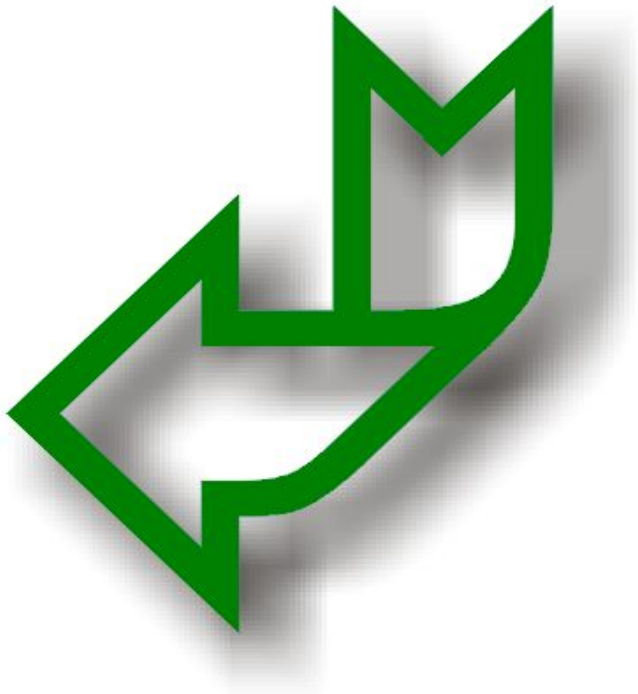
Two forms:

- return without value

```
return;
```

- return with value

```
return 0;
```



# Class Tasks

1. Write a java code to check whether the entered year is a leap year or not.
2. Write a java code to check whether the entered number is **POSITIVE, NEGATIVE or ZERO.**
3. Write java program that inputs three numbers and displays the smallest number.
4. Write a java program to print the month for the given number.
5. Draw a given pyramid using for loop.